

Introdução ao gerenciamento de memória

Samuel Francisco Ferrigo

Introdução

Outra importante função do SO é coordenar e gerenciar a utilização dos diversos tipos de memória (cache, principal, secundária, etc) de forma eficiente

Ele faz isso por meio do gerenciador de memória

Esse gerenciador controla as partes da memória em uso e livres, alocando memória quando os processos precisam e liberando-a quando terminam

Introdução

Os primeiros computadores não possuíam abstração de memória

O processo do usuário era executado diretamente na memória física do computador.

- Em alguns SO embarcados, esse padrão ainda é utilizado

Somente um processo é executado por vez (normalmente)

- Paralelismo viável somente através de threads



Algumas opções de utilização da memória sem qualquer tipo de abstração

Ambos programas (a) e (b) apontam para posições da memória real

0 16380

...

ADD	28
MOV	24
	20
	16
	12
	8
	4
JMP 24	0

(a)

0 16380

...

CMP	28
	24
	20
	16
	12
	8
	4
JMP 28	0

(b)

0 32764

...

CMP	16412
	16408
	16404
	16400
	16396
	16392
	16388
JMP 28	16384
0	16380

...

ADD	28
MOV	24
	20
	16
	12
	8
	4
JMP 24	0

Programa (a) pode estar na memória física, enquanto (b) em outra região da memória física ou na memória secundária

O problema de paralelismo de processos sem abstração de memória, com referência da memória real

Qual o problema dessa solução?

A solução: abstração da memória principal

A abstração da memória é um recurso onde a memória física não é exposta diretamente para o processo

Para isso, cria-se um espaço de endereçamento

- Um conjunto de endereços que cada processo pode usar para endereçar a memória
- Cada processo tem seu espaço de endereçamento
 - Como os telefones, que contém 8 dígitos
 - Servidores DNS, com os espaços separados por pontos

Porém, como diferenciar as posições do espaço de endereçamento de cada processo, com as posições reais da memória física?

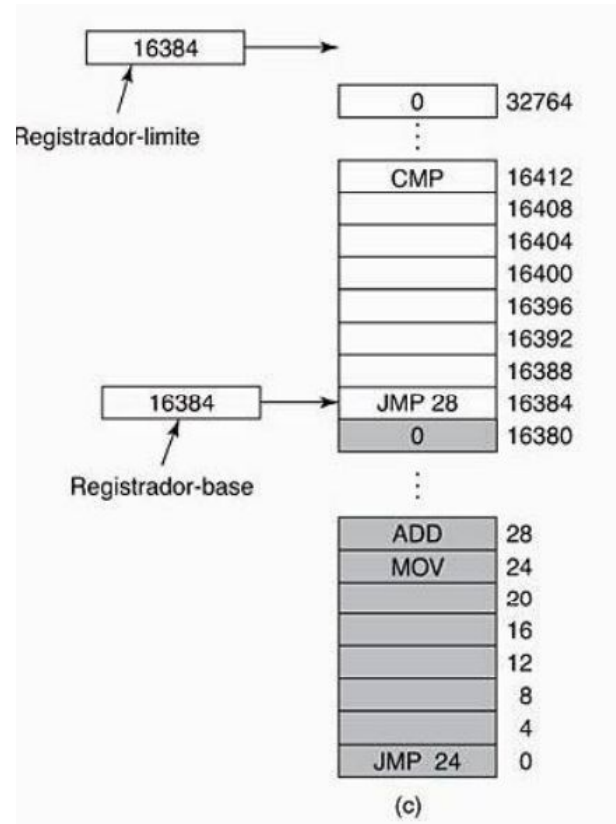
Solução simples: uso de registradores especiais

A primeira solução é o uso de registradores especiais que delimitam a posição de memória inicial e final de um processo

Esses registradores são chamados de registrador-base e registrador-limite, acessíveis apenas pelo modo núcleo

Nessa solução os programas devem estar localizados em posições consecutivas da memória

- Ou seja, não fragmentados



Registrador-base e registrador-limite

Fragmentação

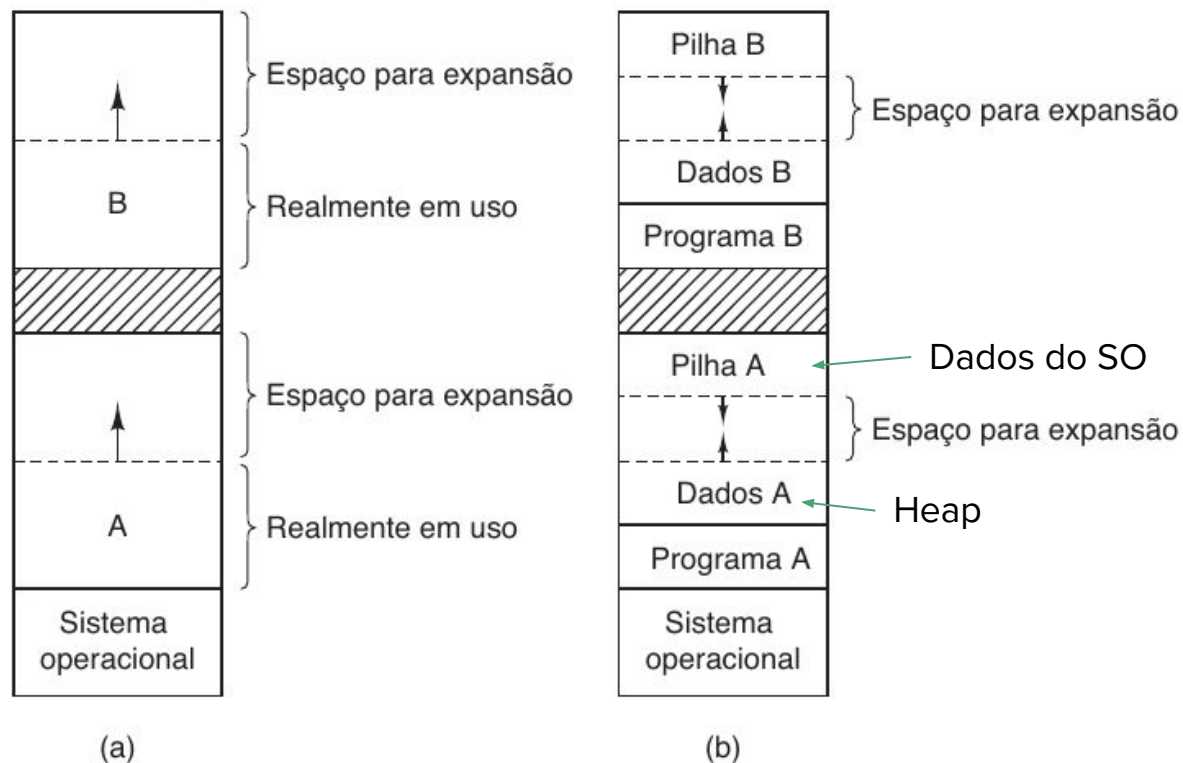
Fragmentação interna

- Quando há espaço livre da memória dentro de um processo

Fragmentação externa

- Quando há espaço livre não contíguo entre processos

(a) Alocação de espaço para um segmento de dados em expansão. (b) Alocação de espaço para uma pilha e um segmento de dados em expansão.



Alocação de espaço pelos processos e as respectivas fragmentações

Troca de memória (swap)

Permite que a quantidade de memória necessária para execução dos processos seja maior que a memória física disponível

Consiste em trazer todo o processo para a memória principal, executá-lo e, então, devolvê-lo ao disco

Útil para processos ociosos, que são executados poucas vezes

Gerenciamento da memória livre

Introdução

Uma das tarefas do Sistema Operacional é lidar com o gerenciamento do espaço livre disponível

Como os processos são dinâmicos, o uso da memória muda constantemente e cabe ao SO gerenciá-la

Técnicas de gerenciamento do espaço livre

Mapas de bits

- A memória é dividida em unidades de alocação, e cada bit demonstra se essa unidade está ocupada ou livre

Listas encadeadas

- Mostram os espaços livres e ocupados por meio de lista(s) encadeada(s)
 - As listas podem alternar espaços livres ou ocupados
 - Serem simples ou duplamente encadeadas
 - Ou trabalharem com listas distintas de espaços livres e ocupados da memória
 - Neste último caso, trata-se de uma solução que agiliza a inserção de um novo processo na memória, pois busca-se apenas na lista de espaços livres
 - Porém requerem maior esforço para fazer a atualização das listas

Algoritmos para gerenciamento da memória livre

Primeiro encaixe (first fit)

Próximo encaixe (next fit)

Melhor encaixe (best fit)

Pior encaixe (worst fit)

Rápido encaixe (quick fit)

Primeiro encaixe (first fit)

É o mais simples

Rápido

Inicia a procura pela posição inicial da lista

Procura o primeiro espaço livre que comporta o processo

Próximo encaixe (next fit)

Semelhante ao first fit, porém inicia a procura a partir do ponto da memória em que encaixou o último processo

Simulações feitas desde a década de 1970 mostram que tem um desempenho ligeiramente pior que o first fit

Melhor encaixe (best fit)

Busca em toda a lista o espaço livre mais próximo ao tamanho estimado do processo

É mais lento que os algoritmos anteriores

Resulta em um desperdício maior da memória, pois sobram espaços livres pequenos demais para comportar novos processos

- Esse desperdício é a fragmentação externa

O first fit gera espaços livres maiores em média

Pior encaixe (worst fit)

Percorre toda a lista e escolhe o maior espaço livre

Tenta assegurar que o segmento livre restante seja útil para alocação de novos processos

Rápido encaixe (quick fit)

Mantém listas em separado para espaços livres de memória mais comumente solicitados

Facilita a busca por espaço livre

Dificulta a busca para a junção de processos próximos, fragmentando rapidamente a memória